WebClient Computing

Javascript Basics

Department of Computer Science

Kyungyong Lee

Operators in Javascript

- Relational operator
  - An operator to express relation between two variables
  - Return value is true or false
  - > : greater than
  - < : less than
  - >= : greater than or equal to
  - <= : less than or equal to
  - == : same
  - != : different

Relational Operator Example

- rel_op.js

```
var a = 123;
var b = 456;
var c = 123;

var a_gt_b = a > b;
console.log(a_gt_b);
console.log(a < b);
console.log(a < c);
console.log(a <= c);
console.log(a == b);
console.log(a == c);
console.log(a != b);
console.log(a != c);
```

```
false                    rel_op.js:6
true                     rel_op.js:7
false                    rel_op.js:8
true                     rel_op.js:9
false                    rel_op.js:10
true                     rel_op.js:11
true                     rel_op.js:12
false                    rel_op.js:13
```

Boolean Values

- Value to represent **true** or **false**
  - By comparison (boolean.js)

```
var a = 123;
console.log(a == 123);

var b = 456;
console.log(b == 456);

var c = "123";
console.log(a == c);
console.log(a === c);
```

  - Boolean() function: > Boolean(10);Boolean(0);

```
> Boolean(10)
<· true
> Boolean(0)
<· false
> Boolean("string")
<· true
> Boolean(null)
<· false
```

Javascript Boolean

- ==
  - Automatic type conversion and matches for the value
- ===
  - Match the value and type (class)

```
var a = 123;
console.log(a == 123);

var b = 456;
console.log(b == 456);

var c = "123";
console.log(a == c);
console.log(a === c);
```

```
true                    boolean.js:2
false                   boolean.js:5
true                    boolean.js:8
false                   boolean.js:9
```

Logical Operator

- Check for given conditions and returns true or false
  - Age is over 20 and last name is Kim
  - Lives in Seoul or Lives in Busan
  - Cf. relational operator expresses relation between two variable (greater than, …)
- And
  - &&
- Or
  - ||
- Not
  - !

Logical Operator And

- And operator - &&
- Evaluated with two variable – A && B
- When condition of both a and b are true, output is true
  - A = true; B = true; A&&B → true
  - A = true; B = false; A&&B → false
  - A = false; B = true; A&&B → false
  - A = false; B = false; A&&B → false

Logical And Example

- logical_and.js

```
var a = true;
var b = false;

console.log(a && a);
console.log(a && b);
console.log(b && a);
console.log(b && b);
```

```
true                    logical_and.js:4
false                   logical_and.js:5
false                   logical_and.js:6
false                   logical_and.js:7
```

Logical Or

- Or operator - ||
- Evaluated with two variables – a || b
- When condition of either a or b is true, the output is true
  - A = true; B = true; A || B → true
  - A = true; B = false; A|| B → true
  - A = false; B = true; A||B → true
  - A = false; B = false; A||B → false

Logical Or Example

- logical_or.js

```
var a = true;
var b = false;

console.log(a || a);
console.log(a || b);
console.log(b || a);
console.log(b || b);
```

```
true                                    logical_or.js:4
true                                    logical_or.js:5
true                                    logical_or.js:6
false                                   logical_or.js:7
```

Logical Not Operator

- Not operator - ! (exclamation mark – 느낌표)
- Negates the current boolean value
  - True → False
  - False → True
- Operates with a single variable and locates before the variable
  - !variable

Logical Not Operator

- logical_not.js

```
var a = true;
var b = false;

console.log(!a);
console.log(!b);
console.log(!!a);
console.log(!!b);
```

```
false                                   logical_not.js:4
true                                    logical_not.js:5
true                                    logical_not.js:6
false                                   logical_not.js:7
```

Priorities in Operators

- How to handle when multiple operators are in one line
  - 2*3 > 4+5 && 6 /3 == 2 || !false
- Priority in order
  - Logical Not operator (!)
  - Arithmetic operator (* / %) → left to right
  - Relational operator (< > <= >= != ==)
  - Logical operator and (&&)
  - Logical operator or (||)
- Note that parenthesis (괄호) always take the highest priority

Operator Priority in Action

- 2*3 > 4+5 && 6 /3 == 2 || **!false**
- **2*3** > **4+5** && **6 /3** == 2 || true
- **6 > 9** && **2==2** || true
- **false && true** || true
- false || true
- true
- Is this a good expression? NOPE
  - Too complicated to understand
  - Avoid making such statements
  - If you have to, use parenthesis where necessary

Using Parenthesis

- age < 27 && gender == “male” || age < 24 && gender == “female”
- Better to use parenthesis
- (age < 27 && gender == “male”) || (age < 24 && gender == “female”)

Using Operator Example

- operator_priority.js

```
var a = 2*3 > 4+5 && 6/3 == 2 || !false;
console.log(a);

var b = (((2*3) > (4+5)) && (6/3)) || !false;
console.log(b);

var c = 2 + 3 * 5;
console.log(c);

var d= (2 + 3) * 5;
console.log(d)
```

true	operator_priority.js:2
true	operator_priority.js:5
17	operator_priority.js:8
25	operator_priority.js:11

Array in Javascript

- Array
  - A collection of data values
  - Cf. Object – each data value in the object has a name (key) – “id”: “leeky”
  - Data values in array has an index
  - Different data type can be in an array
  - Define
    - var a = new Array();
    - var a = [];

Array Exercise

- array.js

```
var arr1 = [];
var arr2 = [1, 2, 3, 4, 5];
var arr3 = [1, "string", true, {x:1, y:3}, [1, 2]]
var arr4 = new Array();
arr4[0] = "array-4";
```

```
> arr1
<· []
> arr2
<· [1, 2, 3, 4, 5]
> arr3
<· [1, "string", true, ▶Object, ▶Array[2]]
> arr4
<· ["array-4"]
> arr3[0]
<· 1
> arr3[3]
<· Object {x: 1, y: 3}
> arr3[4]
<· [1, 2]
>
```

Array Operation

- pop()
  - Remove an element from the end
- shift()
  - Remove an element from the beginning
- push(data)
  - Insert data at the end of an array
  - Returns a size of array
- unshift(data)
  - Insert data at the beginning of an array
  - Returns a size of array

Array Operation

- reverse()
  - Make the ordering of an array to the opposite (last one to first, …)
- sort()
  - Sort in the ascending order
- concat(arr2)
  - Combine two array
- indexOf(value), lastIndexOf(value)
  - Index within an array for the given value

Array Operations Example

- array_operation.js

```
var arr1 = [1, 2, 3, 4, 5];
var arr2 = [1, 2, 3, 4, 5];
```

```
> console.log(arr1);
  [1, 2, 3, 4, 5]
<· undefined
> arr1.pop();
<· 5
> arr1.shift();
<· 1
> console.log(arr1);
  [2, 3, 4]
<· undefined
> arr1.push(1);
<· 4
> arr1.unshift(5);
<· 5
> console.log(arr1);
  [5, 2, 3, 4, 1]
```

```
> arr1.reverse();
<· [1, 4, 3, 2, 5]
> console.log(arr1);
  [1, 4, 3, 2, 5]
<· undefined
> arr1.sort();
<· [1, 2, 3, 4, 5]
> console.log(arr1);
  [1, 2, 3, 4, 5]
<· undefined
> arr1.concat(arr2);
<· [1, 2, 3, 4, 5, 1, 2, 3, 4, 5]
> console.log(arr1);
  [1, 2, 3, 4, 5]
<· undefined
> var arr3 = arr1.concat(arr2);
<· undefined
> arr3.indexOf(2);
<· 1
> arr3.lastIndexOf(2);
<· 6
```

Conditional Statement

- if(boolean)
  - Performs command if the boolean is true
- If(boolean)-else
  - Performs if command if boolean is true. Otherwise, run else commands
- If(boolean-1)-else if(boolean-2)
  - Only if boolean-1 is false and boolean-2 is true, else if command is executed
- switch (condition) – case: break;– default
  - The condition in the switch is evaluated with case to select which command to execute
  - Missing break executes all the case statement until it meets break or the end

Conditional Statement Exercise (if)

- if_statement.js

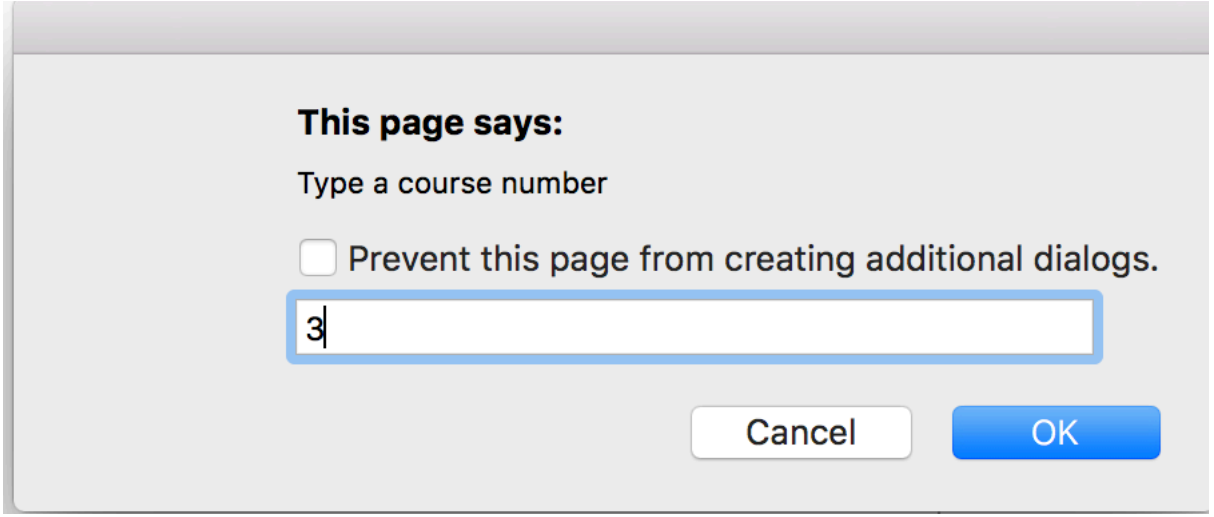
```
1: this is true
2: 1 is NOT larger than 3
3: 3 < 5 || 5 > 7
```

```javascript
if (true) {
  console.log("1: this is true");
}

if (false) {
  console.log("1: this is false");
}

if (1 > 3) {
  console.log("2: 1 is larger than 3");
} else {
  console.log("2: 1 is NOT larger than 3");
}

if (3 < 5 && 5 > 7) {
  console.log("3: 3 < 5 && 5 > 7");
} else if (3 < 5 || 5 > 7) {
  console.log("3: 3 < 5 || 5 > 7");
} else {
  console.log("3: none of the above");
}
```

Conditional Statement Example (switch)

- switch_statement.js

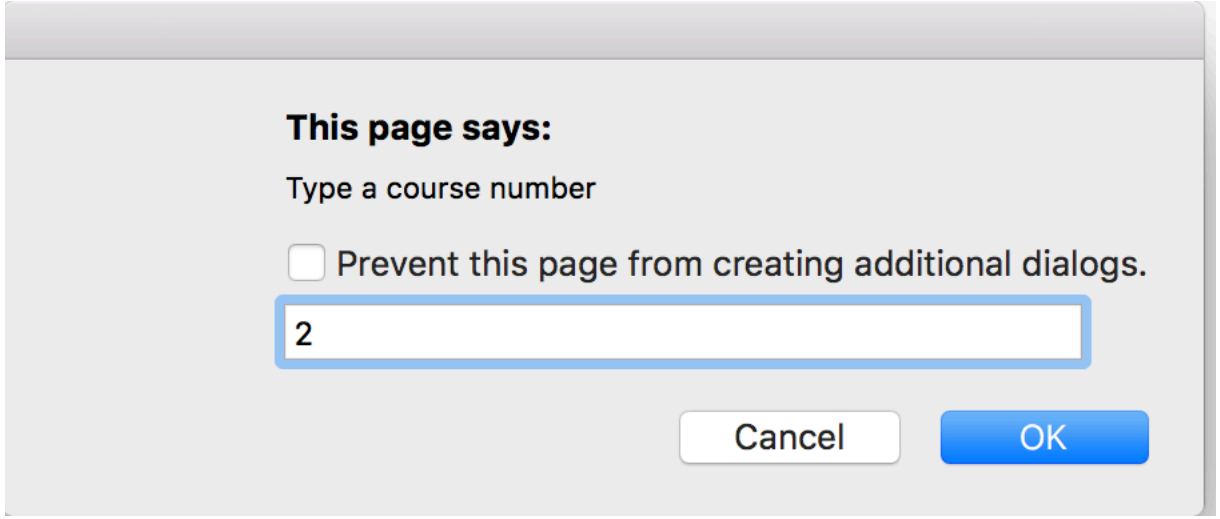
```js
var input = parseInt(prompt("Type a course number"))

switch(input) {
  case 1:
    console.log("WebClient computing");
    break;
  case 2:
    console.log("WebServer computing");
    break;
  case 3:
    console.log("Data Structure");
    break;
  default:
    console.log("No course");
    break;
}
```




```
Data Structure
```

Switch Statement Missing break

```
var input = parseInt(prompt("Type a course number"))

switch(input) {
  case 1:
    console.log("WebClient computing");
  case 2:
    console.log("WebServer computing");
  case 3:
    console.log("Data Structure");
  default:
    console.log("No course");
}
```




```
WebServer computing
Data Structure
No course
```

Loop in Javascript

- while(condition)
  - Execute commands in the while statement until the condition is true
- while(true) – break and continue
  - Execute commands in the while loop until it meets break. If it meets continue, it restarts from the beginning
- do – while(condition)
  - First execute command in the do statement and check condition in while to check if it will continue to execute

Loop Example

- while.js

```
var arr1 = [10, 5, 9, -3, 4, 5];
var index = 0;

while (arr1[index] > 0) {
  console.log("Positive value: " + arr1[index++])
}

while (true) {
  var sum_input = parseInt(prompt("2 + 5 = "));
  if (sum_input != 7) {
    console.log("a wrong answer. try again");
    continue;
  }

  console.log("pass add question");

  var mul_input = parseInt(prompt("2 * 5 = "));
  if (mul_input == 10) {
    console.log("pass multiplication question");
    break;
  }
  console.log("failed multiplication question.");
}
```

Loop Example

- do_while.js

```
var answer = parseInt(prompt("5 * 7 = "));
while(answer != 35) {
  answer = parseInt(prompt("5 * 7 = "));
}
console.log("You got a correct answer");

do {
  answer = parseInt(prompt("5 * 7 = "));
} while (answer != 35)
console.log("You got a correct answer with do-while");
~
```

For Loop in Javascript

- for (initialize; condition; update)
  - The initialize code always executes
  - If the condition is not true, the body does not execute
  - After the body execution, update is executed
  - break and continue in for loop
    - break – always escape the current for loop
    - continue – perform update and check condition then execute

For Loop Example

```
arr1 with while loop: 1
arr1 with while loop: 3
arr1 with while loop: 5
arr1 with while loop: 7
arr1 with while loop: 9
arr1 with for loop: 1
arr1 with for loop: 3
arr1 with for loop: 5
arr1 with for loop: 7
arr1 with for loop: 9
print even number: 2
print even number: 4
print even number: 6
print even number: 8
```

```javascript
var arr1 = [1,3,5,7,9];
var index = 0;

while (arr1[index] != undefined) {
  console.log("arr1 with while loop: " + arr1[index]);
  index++;
}

for(var index = 0; arr1[index] != undefined; index++) {
  console.log("arr1 with for loop: " + arr1[index]);
}

var arr2 = [1,2,3,4,5,6,7,8,9];
for(var index = 0; ; index++) {
  if (arr2[index] == undefined) {
    break;
  }

  if (arr2[index] % 2 != 0) {
    continue;
  } else {
    console.log("print even number: " + arr2[index])
  }
}
```

Iterate For Loop Using in

- Iterate through object keys using in operator
  - for (var key in object)
    - key is the key of the object

```
var input = {name: "kyungyong", work: "KMU", age: 100};

for(var i in input) {
  console.log("key is: " + i + " value is " + input[i]);
}
```

```
key is: name value is kyungyong
key is: work value is KMU
key is: age value is 100
```

Reference

- Javascript, the definitive guide
- W3Schools Javascript